

PTMS Product Requirements Document v1.2

Table of Contents

1. Overview & Objectives

- 1.1 What PTMS Is
- 1.2 Primary Use Case: QRIS Payments
- 1.3 Core Objectives
- 1.4 Design Principle: Extreme Simplicity for API Consumers
- 1.5 Multi-Tenant Architecture
- 1.6 Technology Stack
- 1.7 Scale Parameters

2. Functional Requirements

- 2.1 Multi-Vendor Routing Engine & QRIS Flow
- 2.2 QRIS Callback Handling
- 2.3 Event Sourcing & CQRS
- 2.4 Transaction Status API
- 2.5 Basket-Size Routing
- 2.6 Vendor Health Monitoring & Circuit Breaker
- 2.7 Penalty-Based Routing (No Transaction Retry)
- 2.8 Rate Limiting & Volume Limits
- 2.9 Configuration & Admin API (Laravel)
- 2.10 Observability & Reporting
- 2.12 Admin Dashboard UI (Laravel Filament)

3. Non-Functional Requirements

- 3.1 Performance
- 3.2 Reliability & Availability
- 3.3 Scalability
- 3.4 Security
- 3.5 Maintainability
- 3.6 Data Retention Policy

4. User Stories

- US-1: Penalty-Based Vendor Routing
- US-2: Basket-Size Cost Optimization
- US-3: Rate & Volume Limit Protection
- US-4: Real-Time Circuit Breaker
- US-5: Dedicated Account Pools per User
- US-6: QRIS Generation and Callback Delivery

5. Success Metrics

6. Out of Scope (v1)

7. High-Level Timeline

- Phase 1 — Foundation (Weeks 1-4)
- Phase 2 — QRIS Implementation & Resilience (Weeks 5-8)
- Phase 3 — Production Hardening (Weeks 9-12)
- Phase 4 — Dashboard, Optimization & Observability (Weeks 13-16)

8. v1.1 Changes Summary

Decision 1: Vendor API Failure → Fail-fast (Option A)

Decision 2: Idempotency → Hash key + request body (Option B)

Decision 3: Currency → IDR-only for v1 (Sage's Recommendation)

Decision 4: Penalty Decay → Persist in PostgreSQL, Calculate on Read

Decision 5: Transaction Expiration → 24h Safety-Net + `expires_at` Field

9. v1.2 Changes Summary

Architecture Split: Go Transaction API + Laravel Admin

PTMS Product Requirements Document v1.2

Document Version: 1.2

Date: 2026-04-18

Status: Final — Ready for Stakeholder Review & Implementation

Changes from v1.0: 5 design decisions incorporated (see [v1.1 Changes Summary](#))

Changes from v1.1: Architecture split — Go Transaction API + Laravel Admin (see [v1.2 Changes Summary](#))

Table of Contents

- [1. Overview & Objectives](#)
 - [2. Functional Requirements](#)
 - [3. Non-Functional Requirements](#)
 - [4. User Stories](#)
 - [5. Success Metrics](#)
 - [6. Out of Scope \(v1\)](#)
 - [7. High-Level Timeline](#)
 - [8. v1.1 Changes Summary](#)
 - [9. v1.2 Changes Summary](#)
-

1. Overview & Objectives

1.1 What PTMS Is

The Payment Traffic Management System (PTMS) is a high-throughput middleware layer sitting between client applications and downstream payment vendors/gateways. It intelligently routes, rate-limits, and safeguards payment transaction traffic at scale.

PTMS is NOT a payment gateway. It routes requests — it does not process transactions, tokenize cards, handle 3DS, or settle funds.

1.2 Primary Use Case: QRIS Payments

PTMS is optimized for QRIS (Quick Response Code Indonesian Standard) payments. The flow:

- Generate:** User calls PTMS API → PTMS routes to optimal vendor → returns QRIS code
- Pay:** User scans QRIS via banking/e-wallet app

3. **Callback:** Vendor notifies PTMS → PTMS forwards normalized callback to user

1.3 Core Objectives

- **(a)** Maximize transaction success rates by routing away from failing vendors in real-time
- **(b)** Protect downstream vendors from overload through per-vendor and global rate limiting
- **(c)** Optimize economics via basket-size-aware routing that factors in vendor cost structures
- **(d)** Provide full observability into routing decisions, vendor health, and system throughput

1.4 Design Principle: Extreme Simplicity for API Consumers

Clients send only `amount` and `payment_channel`. The system handles all vendor-specific requirements automatically.

1.5 Multi-Tenant Architecture

Each user (identified by API key) has their own dedicated pool of vendor accounts. Complete isolation between users — no account overlap.

1.6 Technology Stack

Component	Role
NGINX	Ingress proxy: TLS termination, request buffering, load balancing, API gateway routing (<code>/api/v1/*</code> → Go, <code>/admin/*</code> → Laravel)
Go	Transaction API: core routing engine, QRIS generation, callback handling (concurrency, low-latency)
Laravel 11	Admin service: dashboard (Filament v3), Admin API, configuration management, user/vendor CRUD
Redis	Fast state: rate-limit counters, circuit-breaker state, hot config cache, config pub/sub channel
PostgreSQL	Routing rules, vendor profiles, transaction logs, audit history
RabbitMQ	Async event delivery: fallback queues, webhook dispatch, DLQ
Prometheus/ Grafana	Real-time dashboards and alerting

1.7 Scale Parameters

- **Users:** 50–100 (API keys)

- **Account Pool:** 200 accounts per user per vendor at launch
 - **Minimum Vendors:** At least 2 vendors per user for redundancy
 - **Target Throughput:** 500 TPS steady-state, 800 TPS burst (60s)
 - **Infrastructure:** Cloud-agnostic (AWS, GCP, Huawei Cloud, on-prem K8s)
-

2. Functional Requirements

2.1 Multi-Vendor Routing Engine & QRIS Flow

API Endpoint: RESTful (JSON over HTTPS)

- Required fields: `amount`, `currency` (default: "IDR"), `payment_channel` (default: "qris")
 - **v1.1:** Only "IDR" currency is supported. HTTP 400 returned for other currencies.
- Optional: `idempotency_key` (client-provided UUID)
- Authentication: API key

Idempotency: Client provides `idempotency_key` (UUID) in request header.

- Composite key: `{user_api_key}:{idempotency_key}:{SHA256_hash_of_request_body}`
- Stored in Redis (TTL: 24h)
- Any change to amount, currency, or payment_channel = different idempotency key
- Duplicate keys with same parameters → return cached response
- Duplicate keys with different parameters → return `IDEMPOTENCY_CONFLICT` error

Vendor Parameter Auto-Generation: System automatically generates mandatory vendor parameters (name, email, phone, etc.) using randomized data so callers don't need to provide them.

QRIS Generation Flow:

1. Receive generation request from user
2. Route to optimal vendor based on health, rate limits, and volume limits
3. Call vendor's QRIS generation API with auto-generated parameters
4. **If vendor call fails** (timeout, 5xx, connection error):
 - Apply penalty to failing vendor (+10 for timeout, +15 for 5xx)
 - Return HTTP 503 error to user immediately
 - Do NOT silently failover to next vendor (consistent with no-retry philosophy)
 - User must retry with a new request
5. If vendor call succeeds: return QRIS code/image/URL to user within < 1 second
6. Store transaction state as "pending_payment"

Vendor/Account Management:

- Maintain configurable vendor configurations, each with multiple accounts
- Each account exclusively assigned to one user (API key); no sharing

Enable/Disable Controls:

- Disabled vendors → excluded from routing entirely
- Disabled accounts → excluded from account distribution within vendor
- Disabled users → HTTP 403 (cannot generate QRIS or query status)

No Eligible Vendor Handling: When all vendors disabled/rate-limited/circuit-open → HTTP 503

Structured Error Codes (standardized JSON format):

```
{
  "error": {
    "code": "ERROR_CODE",
    "message": "Human-readable description",
    "retry_after": 30,
    "trace_id": "abc-123"
  }
}
```

Error codes: RATE_LIMITED, VOLUME_EXCEEDED, NO_ELIGIBLE_VENDOR, VENDOR_TIMEOUT, VENDOR_ERROR, CIRCUIT_OPEN, USER_DISABLED, INVALID_AMOUNT, IDEMPOTENCY_REPLAY, IDEMPOTENCY_CONFLICT

Routing Architecture (Two Layers):

Layer 1 — Vendor Selection: Route transactions to vendors based on:

- Basket-size brackets and cost efficiency
- Vendor health status (excluding disabled vendors/accounts)
- Penalty scores (lower penalty = higher priority)
- Rate and volume limits

Layer 2 — Account Distribution (Power of Two Choices): Within a selected vendor, pick 2 random enabled accounts, route to the one with lower current load (in-flight requests + recent latency). Provides better load balancing than round-robin for heterogeneous account performance.

Vendor Eligibility Rules: Supported payment channels, min/max transaction amounts.

2.2 QRIS Callback Handling

Callback Reception: Webhook endpoints for each vendor to send payment completion callbacks.

Callback Validation: Verify callback authenticity using vendor-specific signature/token validation.

Callback Normalization: Convert vendor-specific payloads to standard PTMS format:

```
{
  "transaction_id": "ptms-txn-123",
  "vendor_transaction_id": "vendor-abc-456",
  "status": "paid",
  "amount": 100000,
  "paid_at": "2026-04-14T20:12:00Z",
  "payment_method": "qris",
  "vendor_id": "vendor_a"
}
```

Callback URL Security (SSRF Protection): User callback URLs validated on registration:

- HTTPS only (no HTTP)
- Blocked private/internal IP ranges: 10.x.x.x, 172.16-31.x.x, 192.168.x.x, 169.254.x.x, localhost, 127.0.0.1
- DNS resolution verified at registration time
- Test ping sent to validate endpoint responsiveness
- Double DNS resolution at delivery time (prevent DNS rebinding)

User Callback Forwarding: Forward normalized callbacks to user's registered URL within < 500ms.

Callback Retry Mechanism: If user callback fails → retry with exponential backoff (3 attempts over 5 minutes).

Callback Idempotency: Duplicate vendor callbacks (same transaction ID) handled once and only once.

Dead Letter Queue (DLQ): Failed callbacks (after retries) stored in RabbitMQ DLQ.

- Max retention: 7 days
- Alerting: if callback failure rate > 1% in last hour OR absolute DLQ depth > 1000 messages
- Auto-disable: after 3 consecutive days of callback failures, auto-disable user's callback URL (manual re-enable required)
- Admin can view DLQ contents, inspect, manually retry or discard

Transaction State Management:

- Vendor "paid" callback → status "paid"
- Vendor "expired" callback → status "expired"
- Vendor "failed" callback → status "failed"
- PTMS does NOT auto-expire transactions based on internal timers alone

Safety-Net Expiration (**v1.1 — see [Decision 5](#)):

- Vendor QRIS codes typically expire in 5-15 minutes (vendor-specific)
- PTMS does not enforce vendor expiration windows

- If transaction remains "pending_payment" > 24 hours AND reconciliation sweeper has attempted vendor status check ≥ 3 times with no resolution → mark as "expired_stale" (distinct from vendor-reported "expired")
- Status API includes `expires_at` field when vendor provides expiration time
- Logged as system-initiated action. Alert fires on any `expired_stale` transition

Reconciliation Sweeper: Background job runs every 10–15 minutes (configurable) to query vendor APIs for transactions in "pending_payment" > 30 minutes old. Updates status and forwards callback based on vendor response. Logs all actions for audit.

2.3 Event Sourcing & CQRS

Event Store (Write Model): All transaction state changes stored as immutable events in PostgreSQL (JSONB format).

Event Types:

Event Type	Description
<code>qr_generated</code>	QRIS code created, vendor assigned
<code>qr_generation_failed</code>	Vendor returned error
<code>callback_received</code>	Vendor callback received
<code>callback_validated</code>	Callback signature verified
<code>callback_forwarded</code>	Callback sent to user
<code>callback_delivery_failed</code>	User endpoint rejected callback
<code>status_updated</code>	Transaction status changed (paid/expired/failed)
<code>expired_stale</code>	Safety-net expiration triggered
<code>sweeper_checked</code>	Reconciliation sweeper queried vendor

CQRS Read Model: Materialized current-state table updated asynchronously by event processor.

- Status queries hit read model (not event replay)
- Event replay only for debugging, auditing, or projection rebuilds
- Read model cached in Redis for < 100ms response times

2.4 Transaction Status API

Endpoint: `GET /api/v1/transactions/{transaction_id}/status`

Response Fields:

Field	Description
<code>transaction_id</code>	Unique transaction identifier
<code>status</code>	<code>pending_payment</code> , <code>paid</code> , <code>expired</code> , <code>failed</code> , <code>expired_stale</code>

Field	Description
amount	Transaction amount
payment_channel	Payment method (e.g., "qris")
vendor_id	Vendor handling the transaction
account_id	Specific vendor account used
created_at	QRIS generation timestamp
paid_at	Payment completion timestamp (if paid)
expired_at	Expiration timestamp (if expired)
callback_delivered	Boolean — callback successfully delivered to user
qris_code	QRIS code (only if status is pending_payment)
expires_at	Vendor-provided expiration timestamp (if available)

Auth: API key required (users query only their own transactions)

Response Time: < 100ms p99

Caching: Redis-backed — TTL 30s for pending, 5min for terminal states

2.5 Basket-Size Routing

- Define basket-size brackets in IDR (e.g., 0–50K, 50K–500K, 500K–5M, 5M+). Multi-currency support deferred to v2.
- Map each bracket to preferred vendor/vendor priority order based on cost-efficiency
- Real-time bracket configuration via admin API (no redeployment)
- Log basket-size bracket and routing decision with every transaction

2.6 Vendor Health Monitoring & Circuit Breaker

Health Metrics: Per-vendor tracking of response latency (p50/p95/p99), error rate, timeout rate, HTTP status code distribution.

Circuit Breaker States: Closed (normal) → Open (vendor bypassed) → Half-Open (probing) → Closed (recovered)

- **Open trigger:** Error rate > configurable threshold (default: 5% within 60s sliding window)
- **Half-open probe:** Configurable % of traffic (default: 5%) sent to test recovery
- **Close condition:** N consecutive successful probes
- State exposed in admin dashboard and via API

2.7 Penalty-Based Routing (No Transaction Retry)

No Transaction Retry: When primary routed vendor fails (timeout, error, circuit open) → transaction fails immediately, error returned to user. Transaction is NOT retried against another vendor.

Penalty System:

Error Type	Penalty Points
Timeout	+10
5xx error	+15
Circuit open	+20

Dynamic Routing Adjustment: Next transaction routes to next eligible vendor with lowest penalty score.

Penalty Decay (**v1.1 — see [Decision 4](#)):

- Penalty scores stored in PostgreSQL `vendor_penalties` table (`vendor_id`, `account_id`, `penalty_points`, `last_updated_at`)
- Decay calculated on read: `effective_penalty = MAX(0, stored_penalty - minutes_elapsed_since_last_update)`
- Penalties survive service restarts
- Points decay at -1 per minute to allow vendor recovery

Penalty Events: Published to RabbitMQ for async processing (vendor performance tracking, analytics).

Audit: Idempotency keys and auto-generated vendor parameters logged for audit.

2.8 Rate Limiting & Volume Limits

Multi-Level Limits:

Level	Limit Types
Per-account	TPS, daily transaction count, daily cumulative amount (IDR only in v1)
Per-vendor	Aggregate TPS and volume across all accounts
Per-user	TPS, transaction count, cumulative volume across assigned accounts
Global	Per-API-key and per-payment channel

Implementation:

- Redis-backed sliding-window counters (sub-millisecond overhead)
- Redis-backed sum aggregation for volume tracking
- Count-based and amount-based limits enforced simultaneously
- HTTP 429 with `Retry-After` headers when limits hit
- Burst allowances via configurable token-bucket overflow

2.9 Configuration & Admin API (Laravel)

Managed by: Laravel Admin service (PHP 8.3 / Filament v3)

Managed Entities:

- Vendor profiles: name, channels, cost structure, credentials, enable/disable
- Vendor accounts: multiple per vendor, individual rate/volume limits, weights, health, enable/disable
- User management: create users (API keys), assign account pools, configure limits, enable/disable
- User-account mapping: exclusive assignment, no overlap
- Auto-generation rules: configure which parameters to auto-generate per vendor
- Routing rules: basket-size brackets, vendor priority, account distribution strategies
- Rate/volume limits: per-account, per-vendor, per-user, global (count + amount)
- Circuit-breaker parameters: thresholds per vendor
- Penalty management: view scores, reset to zero, disable accumulation for maintenance, force-set for testing

Persistence & Hot-Reload: All config stored in PostgreSQL via Laravel Eloquent models with audit trail. Laravel writes changes to Redis cache + publishes to `config:update` Redis channel. Go Transaction API subscribes and reloads within 100ms. No restarts required for either service.

RBAC: Admin and Viewer roles using Filament panel authentication with session-based auth.

2.10 Observability & Reporting

Prometheus Metrics: Request count, latency histograms, error rates, circuit-breaker state changes, rate-limit rejections, volume limit rejections, penalty scores, routing decisions — all per vendor, per account, per user.

QRIS-Specific Metrics:

Metric	Target
Generation response time (p50/p95/p99)	< 1s p95
Generation success rate	≥ 99.5%
End-to-end success (gen → paid)	≥ 95%
Callback forwarding latency	< 500ms p95
Callback delivery success rate	≥ 99.9%

Pre-built Grafana Dashboards: Vendor Health, QRIS Performance, E2E Transaction Success, Callback Delivery, Transaction Routing Flow, Rate Limit Status, Basket-Size Distribution.

Transaction-Level Logging: Full routing decision trace (vendor → account → outcome) and callback delivery status in PostgreSQL.

Structured JSON Logging: To stdout for log aggregation.

OpenTelemetry: Distributed tracing across full request lifecycle (client → PTMS → vendor → callback → user). Includes vendor selection reason, account chosen, latency breakdown, callback delivery status.

2.12 Admin Dashboard UI (Laravel Filament)

Laravel Filament admin panel providing:

- **Real-Time Overview:** Live transaction volume, vendor health, circuit breaker states, rate/volume utilization with progress bars, QRIS-specific metrics
 - **Vendor Management:** CRUD vendors/accounts, health history charts, enable/disable
 - **User Management & Account Assignment:** Create users, configure callback URLs (SSRF protection), assign account pools, ensure no overlap
 - **Auto-Generation Configuration:** Configure parameters and patterns per vendor
 - **Account Distribution:** Power of Two Choices visualization, real-time load per account
 - **Routing Rules Editor:** Visual basket-size bracket editor, vendor priority reordering
 - **Rate & Volume Limit Panel:** Configure limits, view utilization with color-coded warnings
 - **Circuit Breaker Dashboard:** States, manual override, threshold configuration
 - **Penalty Score Dashboard:** Real-time scores, breakdown by error type, decay status, trends, admin overrides
 - **Transaction Search & Audit:** Search by ID/user/vendor/account/time/amount/status; full routing trace
 - **QRIS Performance Monitor:** Response times, success rates, end-to-end completion rates
 - **Callback Management:** Pending/failed callbacks, DLQ contents, manual resend
 - **Alerting & Notifications:** Configurable rules, channels (email/Slack/PagerDuty), QRIS-specific alerts
-

3. Non-Functional Requirements

3.1 Performance

Metric	Target
Routing decision latency	< 5ms p99 (excluding vendor response)
QRIS generation response time	< 1s p95 (total: API request → QRIS returned)
QRIS generation success rate	≥ 99.5%
Callback forwarding latency	< 500ms p95
Callback delivery success rate	≥ 99.9%
System throughput	500 TPS sustained, 800 TPS burst (60s)
Redis operations (rate limiting, CB)	< 1ms p99

3.2 Reliability & Availability

- Target uptime: 99.95% ($\leq$ 22 min downtime/month)
- No single point of failure: all components in HA config
- Graceful degradation: Redis unavailable → in-memory rate limiting at 50% limits; PostgreSQL unreachable → route from cached config
- No transaction data loss: all routing decisions persisted async via RabbitMQ to PostgreSQL

3.3 Scalability

- Horizontal scaling of Go workers behind NGINX (stateless; shared Redis state)
- Redis Cluster support for rate-limit/CB state
- PostgreSQL read replicas for reporting; primary for transactional writes

3.4 Security

- TLS 1.2+ for all API traffic
- Admin API: JWT with short-lived tokens; IAM/SSO integration
- Sensitive fields never logged/stored (only routing metadata and transaction references)
- Audit log immutability: append-only in PostgreSQL
- Network segmentation: dedicated VPC subnet; vendor outbound restricted to allowlisted IPs/URLs
- SSRF protection on callback URLs (see [§2.2](#))

3.5 Maintainability

- Docker containerized, Kubernetes orchestrated
- CI/CD with automated unit tests, integration tests (vendor mock servers), canary deployments
- Configuration-as-code: routing rules in version-controlled YAML
- Graceful shutdown: drain mode on deployment — stop accepting new requests, allow in-flight calls (10s timeout), then shutdown. K8s preStop hook. NGINX health checks detect draining workers.

3.6 Data Retention Policy

Data Type	Retention
Transaction logs	90 days hot (PostgreSQL) → archived to S3 (Parquet)
Audit trails	7 years (regulatory)
Penalty scores	30 days rolling
Rate limit counters	24 hours rolling
Volume limit counters	Calendar day (midnight Asia/Jakarta)

Implementation: PostgreSQL monthly partitioning with automated archival.

4. User Stories

US-1: Penalty-Based Vendor Routing

As a payment operations manager, **I want** the system to avoid routing to problematic vendors and automatically route to healthier alternatives, **so that** we maintain high transaction success rates.

Acceptance Criteria: When Vendor A returns 5xx or times out (>3s), current transaction fails immediately → error to user. Vendor A receives penalty points (+10 timeout, +15 5xx). Next transaction routes to Vendor B (lowest penalty). Penalty points decay at −1/minute (see [§2.7](#)).

US-2: Basket-Size Cost Optimization

As a finance lead, **I want** high-value transactions routed to vendors with lower percentage fees, **so that** we minimize processing costs.

Acceptance Criteria: Transactions > \$5,000 route to Vendor C (0.5% fee) over Vendor A (2.9% fee) when healthy. Bracket mapping configurable via admin API. Routing decisions logged with bracket, vendor, account.

US-3: Rate & Volume Limit Protection

As a vendor relationship manager, **I want** per-account, per-user, and per-vendor rate and volume limits, **so that** we don't exceed contractual caps.

Acceptance Criteria:

- Account D-1's 50 TPS limit reached → route to another account in pool
- Account D-1's daily volume limit (e.g., 5B IDR) reached → route to another account
- User X's aggregate daily volume limit reached → HTTP 429
- Vendor D aggregate 100 TPS reached → HTTP 429 within 2ms
- Grafana alerts at 80% utilization

US-4: Real-Time Circuit Breaker

As an SRE, **I want** automatic stop-routing to degraded vendors, **so that** we don't waste time on failing vendors.

Acceptance Criteria: Vendor B error rate > 5% in 60s → circuit opens → traffic reroutes. After 30s, 5% probes. 10 consecutive successes → circuit closes. All transitions emit Prometheus metrics and RabbitMQ events.

US-5: Dedicated Account Pools per User

As a platform admin, **I want** each user to have their own dedicated account pool per vendor, **so that** transactions don't mix and usage is trackable per user.

Acceptance Criteria:

- User A: 500 accounts on Vendor VA, 500 on Vendor VB (exclusive)
- User B cannot access User A's accounts
- Volume/limits tracked per user across account pool
- Dashboard shows assignments and utilization per user

US-6: QRIS Generation and Callback Delivery

As an end user, **I want** fast QRIS generation and payment callbacks, **so that** I complete transactions smoothly.

Acceptance Criteria:

- QRIS generation < 1s in 99.5% of requests
- Generated QRIS valid and scannable
- Vendor callback received within 30s of payment
- PTMS forwards callback to user < 500ms
- Failed callbacks: 3 retries with exponential backoff → DLQ
- Webhook includes transaction ID, amount, timestamp

- Status only updates from vendor callbacks (no auto-expiry)

5. Success Metrics

Metric	Target	Measurement
QRIS generation response time (p95)	< 1s	<code>ptms_qris_generation_duration_seconds</code>
QRIS generation success rate	$\geq 99.5\%$	<code>ptms_qris_generation_success_total / ptms_qris_generation_total</code>
Routing decision latency (p99)	< 5ms	<code>ptms_routing_duration_seconds</code>
E2E QRIS success rate (gen → paid)	$\geq 95\%$	<code>ptms_qris_paid_total / ptms_qris_generated_total</code>
Callback forwarding latency (p95)	< 500ms	<code>ptms_callback_forward_duration_seconds</code>
Callback delivery success rate	$\geq 99.9\%$	<code>ptms_callback_delivered_total / ptms_callback_received_total</code>
Account distribution balance	$\leq 20\%$ variance	<code>ptms_account_tx_count</code> per user per vendor
Volume limit accuracy	< 0.5% variance	Daily reconciliation report
Circuit-breaker response time	< 2s from breach to open	Prometheus gauge + alert
Account pool isolation	100% zero sharing	Weekly audit query
Cost savings (basket-size routing)	$\geq 15\%$ reduction for high-value txns	Monthly PostgreSQL report
System uptime	$\geq 99.95\%$	<code>up</code> metric + synthetic checks
Rate-limit false rejection rate	< 0.1%	Weekly audit log analysis
Stale pending transactions	< 0.01%	<code>ptms_expired_stale_total / ptms_qris_generated_total</code>
Mean time to add/configure vendor account	< 15 minutes	Operational runbook

6. Out of Scope (v1)

- Payment processing / gateway functionality
 - Fraud detection or risk scoring
 - Refund and chargeback handling
 - Multi-region / multi-cloud deployment (single-region for v1)
 - Real-time ML for routing optimization (rule-based for v1)
 - End-user / customer-facing UI (admin dashboard is internal only)
 - Vendor onboarding automation (manual via admin API)
 - Multi-currency support (IDR only; see [Decision 3](#))
-

7. High-Level Timeline

Phase 1 — Foundation (Weeks 1-4)

- Core routing engine (Go): request ingestion, vendor registry, priority routing
- PostgreSQL schema: users, vendors, vendor_accounts, user_account_assignments, routing rules, transaction logs, audit trail
- Redis: rate-limit counters, volume trackers, circuit-breaker state
- User management and dedicated account pool assignment
- Admin API (Laravel/Filament): basic CRUD scaffolding
- Basic Prometheus metrics
- **Milestone:** Route transactions with rate/volume limiting. Laravel admin shell ready. Internal dogfooding.

Phase 2 — QRIS Implementation & Resilience (Weeks 5-8)

- QRIS generation flow: vendor API integration, response time optimization
- Callback handling: webhook endpoints, validation, user forwarding
- Multi-vendor routing with cost-optimized strategies
- Basket-size routing with configurable brackets
- Circuit breaker (closed → open → half-open → closed)
- Penalty-based routing with idempotency (no retry; penalties influence future routing)
- RabbitMQ: fallback events, callback retry queue, DLQ, async persistence
- Laravel admin: vendor/account/user management Filament resources, routing rules UI, config sync to Redis
- **Milestone:** Full QRIS flow operational. Multi-vendor penalty routing active. Admin dashboard functional. Simulated failure testing passed.

Phase 3 — Production Hardening (Weeks 9-12)

- NGINX ingress: TLS termination, request buffering, load balancing (routing `/api/v1/*` → Go, `/admin/*` → Laravel)
- HA deployment: Redis Sentinel/Cluster, PostgreSQL replication, RabbitMQ mirrored queues, Laravel queue workers
- RBAC on Laravel admin with Filament authentication and session management
- Grafana dashboards and alerting (PagerDuty)
- Load testing: 500 TPS sustained, 800 TPS burst
- Chaos engineering: vendor failure injection, Redis failover, network partition
- **Milestone:** Production-ready. Staged rollout 10% → 50% → 100%.

Phase 4 — Dashboard, Optimization & Observability (Weeks 13-16)

- Admin Dashboard (Filament): completion of advanced panels (circuit breakers, penalties, DLQ management)
 - Performance tuning from production load-test data
 - Operational runbook and on-call documentation
 - Cost-savings reporting (basket-size routing impact)
 - Post-launch retrospective and v2 backlog
 - **Milestone:** Steady state with full admin dashboard. v1 closed out.
-

8. v1.1 Changes Summary

This section documents the 5 design decisions confirmed by Hau and incorporated into this version. Each decision is referenced inline throughout the document.

Decision 1: Vendor API Failure → Fail-fast (Option A)

When: Vendor QRIS generation fails (timeout, 5xx, connection error).

Behavior:

- Immediately return HTTP 503 error to user with vendor error details
- Apply penalty to failing vendor
- Do NOT silently failover to next vendor
- User must retry with a new request
- Consistent with the "no retry" philosophy

Impacted sections: [§2.1](#) (QRIS Generation Flow step 4), [§2.7](#) (No Transaction Retry), [US-1](#).

Decision 2: Idempotency → Hash key + request body (Option B)

Key format: {user_api_key}:{idempotency_key}:{SHA256_hash_of_request_body}

Rules:

- Any change to amount, currency, or payment_channel = different idempotency key
- New error code: `IDEMPOTENCY_CONFLICT` — returned when same idempotency key is used with different request parameters
- Prevents accidental response corruption

Impacted sections: §2.1 (Idempotency), [Error Codes](#).

Decision 3: Currency → IDR-only for v1 (Sage's Recommendation)

Behavior:

- Currency field accepted for forward compatibility, but only "IDR" is valid
- HTTP 400 returned for any currency other than "IDR"
- Volume limits and basket-size brackets are IDR-only (no conversion logic needed)
- Multi-currency support deferred to v2

Impact: Significantly simplifies the implementation. No currency conversion logic, no multi-currency rate limiting, no exchange rate integration.

Impacted sections: §2.1 (API Endpoint), §2.5, §2.8, §6 (Out of Scope).

Decision 4: Penalty Decay → Persist in PostgreSQL, Calculate on Read

Storage: `vendor_penalties` table in PostgreSQL.

Schema: `vendor_id`, `account_id`, `penalty_points`, `last_updated_at`.

Decay formula: `effective_penalty = MAX(0, stored_penalty - minutes_elapsed)`.

Behavior:

- Update `last_updated_at` whenever penalty is modified
- Penalties survive service restarts
- Decay rate: −1 point per minute

Impacted sections: §2.7 (Penalty Decay), §3.6 (Penalty scores: 30-day rolling retention).

Decision 5: Transaction Expiration → 24h Safety-Net + `expires_at` Field

Behavior:

- Keep 24-hour safety-net expiration (`expired_stale` status)

- Reconciliation sweeper checks transactions > 30 min old every 10–15 min
- Add `expires_at` field to status API response when vendor provides expiration time
- QRIS validity is vendor-specific (typically 5–15 min) — documented but not enforced by PTMS
- PTMS does NOT enforce vendor expiration windows, only the 24h safety-net

Impacted sections: [§2.2](#) (Safety-Net Expiration), [§2.4](#) (Response Fields — `expires_at`).

9. v1.2 Changes Summary

Architecture Split: Go Transaction API + Laravel Admin

Date: 2026-04-18

Rationale: Optimize for different concerns — high-throughput transactions vs. rapid admin development

Changes Introduced in v1.2

Aspect	v1.1 (Monolithic Go)	v1.2 (Split Architecture)
Transaction API	Go handles everything	Go handles only transaction API (500 TPS)
Admin API & Dashboard	Go + React	Laravel + Filament
Team Structure	Go backend (4) + Frontend (1)	Go backend (3) + Laravel full-stack (1)
Config Management	Go reads/writes all	Laravel writes config → Redis → Go reads

Technical Details

Go Transaction API:

- High-performance QRIS generation and callback handling
- Routing engine, rate limiting, circuit breaker
- Event sourcing for transaction logs
- 500 TPS sustained, 800 TPS burst

Laravel Admin Service:

- Admin Dashboard UI using Filament v3
- Configuration management (vendors, routing rules, rate limits)
- User/vendor CRUD operations
- Reporting and audit views

- Redis pub/sub for config sync to Go

Config Sync Flow:

Laravel Admin → PostgreSQL (config tables) → Redis (cache + pub/sub) → Go API

Impacted Sections

- §1 (Overview): Updated tech stack
- §2.9 (Configuration & Admin API): Now Laravel responsibility
- §2.10 (Admin Dashboard UI): Now Laravel Filament
- §2.12: New "Config Management Flow" section
- §7 (Timeline): Updated with Laravel milestones

End of PTMS PRD v1.2